

Continuous Deployment (CD)

1. Overview

Continuous Deployment (CD) is a software delivery practice where validated code changes are automatically deployed to production frequently and safely.

Goal:

None

Build → Test → Deploy → Monitor

with minimal manual effort.

CD improves:

- Maintainability
- Release speed
- Reliability
- Rollback capability
- Developer productivity

2. Continuous Delivery vs Continuous Deployment

Term	Meaning
Continuous Integration (CI)	Frequent code merge + automated tests

Continuous Delivery	Code always deployable, manual approval possible
Continuous Deployment	Every approved build auto-deploys to production

Many people confuse delivery and deployment.

3. Why CD Matters in System Design

Without CD:

None

Monthly releases

Manual deploys

Risky big changes

Slow rollback

With CD:

None

Small frequent releases

Low-risk changes

Fast rollback

Quick feedback

4. Key Benefits

A. Faster Feature Delivery

Ship improvements continuously.

B. Smaller Changes = Lower Risk

Small releases easier to debug.

C. Fast Rollback

Bad release? Revert quickly.

D. Better Maintainability

Frequent updates prevent stale systems.

E. Developer Velocity

Less release bottleneck.

5. Typical CD Pipeline

None

Developer Pushes Code



Build



Unit Tests



Integration Tests



Security Checks



Artifact Creation



Deploy Staging



Smoke Tests



Deploy Production



Monitor Metrics

6. Why Small Frequent Deployments Are Better

Compare:

Large Monthly Release

None

500 changes together

Hard to isolate bugs.

Daily Small Releases

None

5 changes each deploy

Much safer.

7. Rollback Strategy

If deployment causes issue:

None

Error rate spikes

Latency rises

Users impacted

Then:

- Revert previous version
- Disable feature flag
- Route traffic back

Fast rollback is core CD strength.

8. Blue/Green Deployment

Two production environments:

- Blue = current live version
- Green = new version

Flow:

None

Deploy new version to Green

Test Green

Switch traffic Blue → Green

If issue:

None

Switch back to Blue

Benefits:

- Near zero downtime
- Fast rollback
- Safer releases

9. Canary Deployment

Release to small percentage first.

Example:

None

1% users

10% users

50% users

100% users

Watch metrics before full rollout.

Best for high-scale systems.

10. Rolling Deployment

Replace instances gradually.

None

10 servers total

Update 2 at a time

Keeps service available.

11. Zero Downtime Deployment

Goal: Users should not notice deployment.

Needs:

- Multiple instances
- Load balancer draining
- Backward compatible changes
- Health checks

12. Database Migration Challenge

Deploying code is easy; schema changes harder.

Need strategies:

Expand / Contract Pattern

Step 1:

Add new column/table compatible with old app.

Step 2:

Deploy new app.

Step 3:

Migrate data.

Step 4:

Remove old schema later.

Avoid breaking running versions.

13. Feature Flags

Deploy code but hide feature.

None

Feature deployed = yes

Visible to users = no

Then enable gradually.

Benefits:

- Safer releases
- A/B testing
- Instant disable without redeploy

14. Monitoring After Deployment

Must track:

- Error rate
- Latency
- CPU/memory
- Conversion metrics
- Crash rate
- DB load

If bad trend appears → rollback.

15. Static Code Analysis

Tools analyze code quality before deploy.

Examples:

- SonarQube
- Linters
- Security scanners

Checks:

- Bugs
- Vulnerabilities
- Duplicated code
- Maintainability issues

- Code smells

16. Security in CD Pipeline

Include:

- Dependency scanning
- Secret detection
- Container scanning
- SAST
- License checks

Shift-left security.

17. Microservices Advantage

CD easier with microservices because:

- Smaller deploy units
- Independent release cycles
- One service update doesn't require whole monolith deploy

But introduces multi-service coordination complexity.

18. Monolith vs Services for Deployment

Aspect	Monolith	Microservices
Single deploy unit	Yes	No

Release independently	Harder	Easier
Blast radius	Large	Smaller
Operational complexity	Lower	Higher

19. Risks of CD

Poor tests can auto-deploy bugs
Schema mismatch issues
Hidden config drift
Too many releases without observability
Broken rollback process

CD needs discipline.

20. Interview Answer Example

Q: How would you deploy with minimal downtime?

Answer:

I'd use blue/green or rolling deployments behind a load balancer, run health checks, keep schema backward compatible, monitor error rates post-release, and maintain instant rollback capability.

21. Best Practices

- Automate everything possible
- Keep deployments small and frequent
- Use canary or blue/green
- Version artifacts immutably
- Monitor every deploy
- Test rollback regularly
- Use feature flags
- Keep DB migrations backward compatible

22. Real-World Pipeline Tools

Examples:

- GitHub Actions
- GitLab CI/CD
- Jenkins
- Argo CD
- CircleCI

23. Short Revision Notes

None

Continuous Deployment:

Auto deploy tested code frequently

Benefits:

- Fast releases
- Easy rollback
- Better maintainability

Strategies:

- Blue/Green
- Canary
- Rolling deploy

Needs:

- Strong tests
- Monitoring
- Safe DB migrations